

FluxMoney + Nimble — API do Assistente Financeiro para WhatsApp

Contrato técnico e guia de configuração da integração.

Versão da documentação: 24/07/2026

Código de referência revisado: API publicada até o endpoint `resolve_transaction`
+ correção de filtros de 24/07/2026

Base URL de produção:

`https://app.fluxmoneyapp.com.br/api/v1/whatsapp`

Todas as operações usam o mesmo endpoint e informam a action na query string:

`https://app.fluxmoneyapp.com.br/api/v1/whatsapp?action=<ACTION>`

Correção crítica desta versão: as actions de criação fazem parte do contrato oficial. A integração não é somente de consulta. A Nimble deve permitir criar receitas comuns, parceladas e fixas/recorrentes; despesas comuns, parceladas e fixas/recorrentes; transferências; compras comuns e parceladas no cartão; categorias de receita/despesa; e tags pelo WhatsApp.

Correção de filtros: a API agora possui `list_transactions`, exige escopo explícito nas consultas sensíveis e normaliza corretamente o perfil PF/PJ dos cartões. Assim, a ausência de parâmetros não pode mais virar silenciosamente uma consulta `all`.

1. O que a integração permite

A Nimble pode:

- consultar contas, cartões, categorias, tags e perfis;
- criar categoria de receita ou despesa;
- criar tag de cartão;
- lançar receita ou despesa comum;
- lançar receita ou despesa parcelada;
- lançar receita ou despesa fixa/recorrente;
- lançar transferência interna entre contas do mesmo perfil;
- lançar movimento entre PF e PJ;
- lançar compra comum no cartão;
- lançar compra parcelada no cartão;
- listar lançamentos com filtros explícitos por perfil, fonte, período, conta, cartão, tipo, situação, categoria, tag e descrição;
- consultar pendências;
- marcar receita, despesa ou movimento vinculado como pago/recebido;
- consultar faturas;
- pagar integralmente fatura fechada ou atrasada;

- consultar resumo, projeção e análise financeira.

A Nimble não pode:

- editar ou excluir lançamentos;
- desfazer baixa;
- desfazer pagamento de fatura;
- pagar fatura parcialmente;
- editar ou excluir recorrências;
- criar, editar ou excluir contas ou cartões;
- alterar assinatura, usuário ou configurações administrativas;
- enviar `user_id` para identificar o usuário.

1.1 Receitas suportadas

- receita comum: `create_transaction` com `type:"receita"`;
- receita parcelada: `create_installments` com `type:"receita"`;
- receita fixa/recorrente: `create_fixed` com `type:"receita"`;
- categoria de receita: `create_category` com `type:"receita"`;
- baixa como recebida: `settle_transaction`.

Compras em cartão são sempre despesas.

2. Actions oficiais

2.1 Consultas — GET

Action	Finalidade
<code>context</code>	Consultar contas, cartões, categorias, perfis e tags.
<code>list_transactions</code>	Listar lançamentos detalhados com filtros explícitos.
<code>pending_transactions</code>	Consultar uma lista de pendências.
<code>payable_invoices</code>	Consultar faturas e verificar se podem ser pagas pela API.
<code>financial_summary</code>	Consultar saldo, pendências, atrasos e próximos vencimentos.
<code>financial_projection</code>	Consultar projeção financeira.
<code>financial_analytics</code>	Consultar análise de gastos por categoria.

2.2 Consultas auxiliares, criações e mutações — POST

Action	Finalidade	confirmed:true
resolve_transaction	Localizar uma única pendência e retornar <code>selected_transaction.transaction_id</code> . É somente leitura.	Não exigido.
create_category	Criar categoria de receita ou despesa.	Não exigido pelo backend.
create_credit_card_tag	Criar tag de cartão.	Não exigido pelo backend.
create_transaction	Criar receita ou despesa comum.	Obrigatório.
create_installments	Criar receita ou despesa parcelada.	Obrigatório.
create_fixed	Criar receita ou despesa fixa/recorrente.	Obrigatório.
create_transfer	Criar transferência interna ou movimento PF/PJ.	Obrigatório.
create_credit_card_purchase	Criar compra comum no cartão.	Obrigatório.
create_credit_card_installments	Criar compra parcelada no cartão.	Obrigatório.
settle_transaction	Baixar pendência como paga/recebida.	Obrigatório.
pay_credit_card_invoice	Pagar integralmente fatura elegível.	Obrigatório.

Actions antigas que não devem ser configuradas:

- `mark_paid`: depreciada; usar `settle_transaction`;
- `mark_unpaid`: não suportada.

3. Autenticação e identificação do usuário

Todas as chamadas exigem:

Authorization: Bearer <SUPPLIER_API_TOKEN>

Regras obrigatórias:

- O token deve ficar somente na configuração segura da Nimble.
- O token não deve aparecer em prompt, conversa, print, log público ou mensagem ao usuário.
- A Nimble identifica o usuário enviando `whatsapp_phone`.
- A API resolve internamente o usuário vinculado ao telefone.

- A Nimble nunca deve enviar `user_id`, nem em query nem no body.
- Se `user_id` for enviado, a API retorna `USER_ID_NOT_ACCEPTED`.

Use placeholders nesta documentação. Não substitua o token em arquivos que serão compartilhados.

4. Headers

4.1 GET

Authorization: Bearer <SUPPLIER_API_TOKEN>

4.2 POST de criação, baixa ou pagamento

Authorization: Bearer <SUPPLIER_API_TOKEN>
 Content-Type: application/json
 X-Idempotency-Key: nimble:<PROVIDER_MESSAGE_ID>:<ACTION>

Todo POST de criação, baixa ou pagamento exige também no body:

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>"
}
```

4.3 POST `resolve_transaction` (somente leitura)

Authorization: Bearer <SUPPLIER_API_TOKEN>
 Content-Type: application/json

`resolve_transaction` não exige `X-Idempotency-Key`, `provider_message_id` nem `confirmed: true`, porque apenas localiza uma pendência e não altera dados.

5. Idempotência

Todos os POSTs de criação, baixa ou pagamento exigem:

- header `X-Idempotency-Key`;
- `provider_message_id` no body;
- mesmo payload quando a chave for reutilizada.

Comportamento:

- mesma chave e mesmo payload: a API repete a resposta sem duplicar a operação;
- mesma chave e payload diferente: `IDEMPOTENCY_PAYLOAD_MISMATCH`;
- nova intenção do usuário: usar novo `provider_message_id` e nova chave;
- retry técnico da mesma mensagem: reutilizar a mesma chave e o mesmo body.

Exceção: `resolve_transaction` é um POST somente de leitura e não exige idempotência.

Padrão recomendado:

`nimble:<provider_message_id>:<action>`

6. Regra de confirmação

Antes de qualquer criação financeira, baixa ou pagamento, a Nimble deve:

1. reunir os dados necessários;
2. apresentar um resumo claro ao usuário;
3. aguardar confirmação explícita;
4. somente então chamar o POST com `confirmed:true`.

Exemplo:

Usuário: Lança R\$ 89,90 de internet no Nubank PF.

Nimble: Confirma lançar uma despesa de R\$ 89,90, descrição Internet, categoria Moradia, na conta Nubank PF, com vencimento em 24/07/2026?

Usuário: Confirmo.

Se a chamada financeira for feita sem `confirmed:true`, a API retorna:

```
{
  "ok": false,
  "error": {
    "code": "CONFIRMATION_REQUIRED",
    "message": "Confirme com o usuário antes de executar esta ação financeira."
  }
}
```

Criar categoria e tag não exige `confirmed:true` tecnicamente, mas a Nimble só deve criá-las quando o usuário tiver solicitado ou aceitado o novo nome.

7. Regra operacional obrigatória da Nimble

7.1 Para lançar algo novo

Quando o usuário disser “lança”, “registra”, “cria”, “adiciona” ou equivalente:

1. chamar `context`;
2. identificar a conta ou o cartão correto;
3. localizar a categoria e a tag pedidas;
4. se a categoria não existir, chamar `create_category`;
5. se a tag de cartão não existir, chamar `create_credit_card_tag`;
6. confirmar o lançamento com o usuário;
7. chamar a action financeira de criação apropriada.

7.2 Para pagar ou baixar algo existente

Quando o usuário disser “paga”, “recebi”, “baixa”, “marca como pago” ou equivalente:

1. chamar `pending_transactions`;
2. escolher o item correto;
3. guardar exatamente o `transaction_id` retornado;
4. confirmar com o usuário;
5. chamar `settle_transaction` enviando o mesmo `transaction_id`.

7.3 Regra inegociável sobre IDs

Os IDs retornados por uma consulta precisam ser reaproveitados na action seguinte:

Consulta	Campo que deve ser guardado	Action seguinte
<code>context</code>	<code>accounts[].id</code>	Criações em conta e pagamento de fatura.
<code>context</code>	<code>credit_cards[].id</code>	Criações no cartão.
<code>pending_transactions</code>	<code>transactions[].transaction_id</code>	<code>settle_transaction</code> .
<code>payable_invoices</code>	<code>credit_card_id</code> e <code>ciclo_key</code>	<code>pay_credit_card_invoice</code> .

A Nimble nunca deve chamar `settle_transaction` sem `transaction_id`.

Se a plataforma não conseguir transportar um ID entre a consulta e a confirmação do usuário, a configuração precisa usar uma variável persistente ou uma camada intermediária de resolução. A API não deve adivinhar qual transação baixar.

8. GET context

Use antes de criar lançamentos ou quando houver dúvida sobre conta, cartão, categoria, perfil ou tag.

`GET /api/v1/whatsapp?action=context&whatsapp_phone=<WHATSAPP_PHONE>`

Exemplo de resposta:

```
{
  "ok": true,
  "accounts": [
    {
      "id": "<ACCOUNT_ID>",
      "name": "Nubank PF",
      "bank": "Nubank",
    }
  ]
}
```

```

        "account_type": "Conta Corrente",
        "profile_type": "PF"
    }
],
"credit_cards": [
    {
        "id": "<CREDIT_CARD_ID>",
        "name": "Nubank",
        "issuer": "Nubank",
        "category": "PF",
        "closing_day": 28,
        "due_day": 10,
        "is_active": true
    }
],
"categories": [
    {
        "id": "<CATEGORY_ID>",
        "profile_id": "pf",
        "type": "despesa",
        "name": "Moradia"
    }
],
"profiles": [
    { "id": "pf", "label": "PF" },
    { "id": "pj", "label": "PJ" }
],
"credit_card_tags": [
    {
        "id": "<TAG_ID>",
        "name": "Assinaturas"
    }
]
}

```

Regras para a Nimble:

- usar `accounts[].id`, nunca o nome, nos campos `account_id`;
- usar `credit_cards[].id` em `credit_card_id`;
- categoria deve corresponder ao perfil e ao tipo do lançamento;
- o body das criações recebe o nome da categoria/tag, não o ID;
- não usar nem armazenar `user_id`, mesmo que uma versão do backend ainda o exponha na resposta.

9. POST `create_category`

Cria categoria nova de receita ou despesa.

POST `/api/v1/whatsapp?action=create_category`

Body:

```

{
    "whatsapp_phone": "<WHATSAPP_PHONE>",
    "provider_message_id": "<PROVIDER_MESSAGE_ID>",

```

```

    "profile_id": "pf",
    "type": "despesa",
    "name": "Pets"
}

```

Campos:

Campo	Obrigatório	Regra
profile_id	Sim	pf ou pj.
type	Sim	receita ou despesa.
name	Sim	Nome não vazio.

Comportamento:

- categoria nova: HTTP 201, status:"created";
- categoria já existente no mesmo perfil/tipo: HTTP 200, status:"already_exists";
- a operação é idempotente;
- depois da criação, usar `category.name` no lançamento financeiro.

10. POST create_credit_card_tag

Cria tag disponível para compras no cartão.

POST /api/v1/whatsapp?action=create_credit_card_tag

Body:

```

{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "name": "Streaming"
}

```

Comportamento:

- tag nova: HTTP 201, status:"created";
- tag já existente: HTTP 200, status:"already_exists";
- depois da criação, usar `tag.name` na compra do cartão.

11. POST create_transaction

Cria uma receita ou despesa comum em conta bancária.

POST /api/v1/whatsapp?action=create_transaction

Body:


```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "confirmed": true,
  "type": "despesa",
  "description": "Internet",
  "amount": 89.9,
  "date": "2026-07-24",
  "paid": false,
  "account_id": "<ACCOUNT_ID>",
  "category": "Moradia",
  "payment_method": "pix",
  "spending_type": "variavel",
  "notes": "Criada pelo assistente"
}
```

Campos:

Campo	Obrigatório	Regra
confirmed	Sim	Exatamente true.
type	Sim	receita ou despesa.
description	Sim	Texto não vazio.
amount	Sim	Valor positivo; a API aplica o sinal.
date	Sim	YYYY-MM-DD.
paid	Sim	Booleano.
account_id	Sim	ID retornado por context.
category	Não	Se enviada, deve existir para perfil e tipo.
payment_method	Não	Ver valores aceitos abaixo.
spending_type	Não	variavel, variável, normal ou fixo.
notes	Não	Observação livre.

payment_method aceita:

- pix
- boleto
- dinheiro
- debito
- credito
- transferencia_bancaria
- debito_conta

Use `create_fixed` para uma recorrência real. Em `create_transaction`, `spending_type` : "fixo" apenas classifica um lançamento único.

12. POST create_installments

Cria receita ou despesa parcelada em conta bancária.

POST /api/v1/whatsapp?action=create_installments

Body:

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "confirmed": true,
  "type": "despesa",
  "description": "Notebook",
  "amount": 3600,
  "date": "2026-07-24",
  "paid": false,
  "installments": 12,
  "account_id": "<ACCOUNT_ID>",
  "category": "Equipamentos",
  "payment_method": "boleto",
  "notes": ""
}
```

Regras:

- `amount` é o valor total da compra/receita;
- a API divide o total pelo número de parcelas;
- `installments` deve ser inteiro entre 2 e 120;
- a primeira ocorrência usa o valor de `paid`; as demais são criadas pendentes;
- a descrição recebe (1/12), (2/12) etc.;
- categoria, se enviada, deve existir.

13. POST `create_fixed`

Cria receita ou despesa fixa/recorrente em conta bancária.

POST `/api/v1/whatsapp?action=create_fixed`

13.1 Sem prazo

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "confirmed": true,
  "type": "despesa",
  "description": "Internet",
  "amount": 89.9,
  "date": "2026-07-24",
  "paid": false,
  "deadline_mode": "sem_prazo",
  "account_id": "<ACCOUNT_ID>",
  "category": "Moradia",
  "payment_method": "pix",
  "notes": ""
}
```

13.2 Com prazo

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "confirmed": true,
  "type": "receita",
  "description": "Contrato mensal",
  "amount": 1200,
  "date": "2026-07-24",
  "paid": false,
  "deadline_mode": "com_prazo",
  "end_date": "2027-06-24",
  "account_id": "<ACCOUNT_ID>",
  "category": "Serviços",
  "payment_method": "pix",
  "notes": ""
}
```

Regras:

- amount é o valor mensal, não o total da série;
- deadline_mode é obrigatório: sem_prazo ou com_prazo;
- sem_prazo gera inicialmente uma janela de 12 meses;
- com_prazo exige end_date;
- end_date não pode ser anterior a date;
- limite máximo: 120 meses;
- apenas a primeira ocorrência usa o valor de paid.

14. POST create_credit_card_purchase

Cria compra comum no cartão de crédito.

POST /api/v1/whatsapp?action=create_credit_card_purchase

Body:

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "confirmed": true,
  "description": "CapCut",
  "amount": 59.9,
  "date": "2026-07-24",
  "credit_card_id": "<CREDIT_CARD_ID>",
  "category": "Assinaturas",
  "tag": "Trabalho",
  "spending_type": "variavel",
  "notes": ""
}
```

Campos:

Campo	Obrigatório	Regra
confirmed	Sim	Exatamente <code>true</code> .
description	Sim	Texto não vazio.
amount	Sim	Valor positivo.
date	Sim	YYYY-MM-DD.
credit_card_id	Sim	ID retornado por <code>context</code> .
category	Não	Deve existir como categoria de despesa no perfil do cartão.
tag	Não	Deve existir em <code>credit_card_tags</code> .
spending_type	Não	variavel, variável, normal ou fixo.
paid	Não	Se omitido, assume <code>false</code> .
notes	Não	Observação livre.

Não enviar:

- `account_id`;
- `installments`.

Para compra parcelada, usar `create_credit_card_installments`.

Importante: `spending_type:"fixo"` classifica esta compra como fixa, mas esta action cria somente uma ocorrência. A API atual não possui uma action dedicada para compra fixa mensal recorrente no cartão.

15. POST `create_credit_card_installments`

Cria compra parcelada no cartão.

POST `/api/v1/whatsapp?action=create_credit_card_installments`

Body:

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "confirmed": true,
  "description": "Celular",
  "amount": 2400,
  "date": "2026-07-24",
  "installments": 12,
  "credit_card_id": "<CREDIT_CARD_ID>",
  "category": "Eletrônicos",
  "tag": "Pessoal",
  "notes": ""
}
```

Regras:

- `amount` é o valor total;
- a API divide o valor pelo número de parcelas;
- `installments` deve ser inteiro entre 2 e 120;
- cada parcela é vinculada ao mês de fatura correspondente;
- `category` e `tag`, se enviadas, precisam existir;
- `paid` é opcional e assume `false`;
- não enviar `account_id`.

16. POST `create_transfer`

Cria transferência interna ou movimento entre perfis.

POST `/api/v1/whatsapp?action=create_transfer`

16.1 Mesmo perfil: `PF → PF` ou `PJ → PJ`

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "confirmed": true,
  "description": "Transferência para reserva",
  "amount": 500,
  "date": "2026-07-24",
  "paid": true,
  "from_account_id": "<FROM_ACCOUNT_ID>",
  "to_account_id": "<TO_ACCOUNT_ID>",
  "deadline_mode": "single",
  "notes": ""
}
```

Resultado:

- cria duas pernas vinculadas por `transferId`;
- não entra como receita/despesa financeira entre perfis;
- transferência interna recorrente não é suportada.

16.2 Perfis diferentes: `PJ → PF` ou `PF → PJ`

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "confirmed": true,
  "description": "Pró-labore",
  "amount": 3000,
  "date": "2026-07-24",
  "paid": false,
  "from_account_id": "<PJ_ACCOUNT_ID>",
  "to_account_id": "<PF_ACCOUNT_ID>",
  "deadline_mode": "sem_prazo",
  "from_category": "Pró-labore",
  "to_category": "Pró-labore",
}
```

```

    "notes": ""
  }

```

Resultado:

- cria despesa na origem e receita no destino;
- as pernas compartilham `linkedMovementId`;
- `deadline_mode` pode ser `single`, `sem_prazo` ou `com_prazo`;
- `com_prazo` exige `end_date`;
- categorias são opcionais; sem elas, a API usa Movimento PF/PJ;
- para data futura, a API força o movimento inicial como pendente.

16.3 Proteção contra duplicidade

Antes de criar, a API procura movimentação pendente compatível.

Se encontrar:

```

{
  "ok": false,
  "error": {
    "code": "PENDING_TRANSFER_MATCH_FOUND",
    "message": "Existe movimentação pendente compatível.",
    "details": {
      "candidates": [],
      "next_step": "Use settle_transaction para baixar a pendência, ou envie create_new_confirmed:true para criar mesmo assim."
    }
  }
}

```

A Nimble deve:

1. apresentar os candidatos;
2. perguntar se o usuário quer baixar a pendência ou criar outra;
3. para baixar, usar o `transaction_id` do candidato em `settle_transaction`;
4. para criar mesmo assim, obter nova confirmação e reenviar com:

```

{
  "create_new_confirmed": true
}

```

17. GET pending_transactions

Consulta receitas, despesas, transferências internas e movimentos PF/PJ ainda pendentes.

GET /api/v1/whatsapp?action=pending_transactions&whatsapp_phone=<WHATSAPP_PHONE>

Filtros opcionais:

Campo	Valores
type	receita OU despesa.
account_id	ID da conta da perna.
from_account_id	Conta de origem do movimento.
to_account_id	Conta de destino.
movement_kind	common, internal_transfer OU pf_pj.
description ou q	Trecho da descrição.
date	YYYY-MM-DD.
amount	Valor positivo exato.
limit	1..100; padrão 50.

Exemplo:

```
GET /api/v1/whatsapp?action=pending_transactions&whatsapp_phone=<WHATSAPP_PHONE>&
type=despesa&description=internet&limit=10
```

Resposta:

```
{
  "ok": true,
  "transactions": [
    {
      "id": "<TRANSACTION_ID>",
      "transaction_id": "<TRANSACTION_ID>",
      "type": "despesa",
      "absolute_amount": 89.9,
      "date": "2026-07-24",
      "description": "Internet",
      "category": "Moradia",
      "account_id": "<ACCOUNT_ID>",
      "account_label": "Nubank PF",
      "profile": "PF",
      "status": "due_today",
      "movement_kind": "common",
      "linked_movement_id": null,
      "transfer_id": null,
      "settle_affects_linked_legs": false,
      "settle_confirmation_message": "Confirma marcar a despesa Internet de R$
89,90 como paga?",
      "paid": false
    }
  ]
}
```

status pode ser:

- overdue
- due_today
- future

17.1 POST resolve_transaction (somente leitura)

Localiza uma pendência específica antes da baixa e devolve o identificador diretamente em:

```
$.selected_transaction.transaction_id
```

```
POST /api/v1/whatsapp?action=resolve_transaction
```

```
Authorization: Bearer <SUPPLIER_API_TOKEN>
```

```
Content-Type: application/json
```

Body mínimo:

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "description": "Capcut"
}
```

Filtros opcionais para desambiguação:

```
{
  "amount": 65.90,
  "date": "2026-07-05",
  "type": "despesa",
  "profile_id": "pf"
}
```

Regras:

- pesquisa somente receitas e despesas pendentes elegíveis para baixa;
- ignora maiúsculas, minúsculas e acentos na descrição;
- prioriza correspondência exata normalizada e usa correspondência parcial como fallback;
- usa amount, date, type e profile_id quando enviados;
- não escolhe arbitrariamente o primeiro item;
- rejeita user_id;
- não altera dados;
- não exige X-Idempotency-Key, provider_message_id nem confirmed:true.

Resposta com uma correspondência segura:

```
{
  "ok": true,
  "status": "selected",
  "selection_required": false,
  "selected_transaction": {
    "transaction_id": "<TRANSACTION_ID>",
    "description": "Capcut (tiktok)",
    "amount": 65.90,
    "date": "2026-07-05",
    "type": "despesa",
    "profile_id": "pf",
    "account_id": "<ACCOUNT_ID>",
    "account_name": "Nubank",
    "settle_confirmation_message": "Confirma marcar a despesa Capcut (tiktok) de R$ 65,90 como paga?"
  }
}
```



```
}
}
```

Resposta ambígua:

```
{
  "ok": true,
  "status": "multiple_matches",
  "selection_required": true,
  "message": "Encontrei mais de um lançamento pendente. Qual deles você deseja
    baixar?",
  "matches": []
}
```

Sem correspondência, retorna HTTP 404 com TRANSACTION_NOT_FOUND.

18. GET list_transactions

Lista lançamentos detalhados e separa corretamente despesas de contas e de cartões. Esta é a action indicada para pedidos como “mostre minhas despesas de cartão PF em julho”.

```
GET /api/v1/whatsapp?action=list_transactions&whatsapp_phone=<WHATSAPP_PHONE>&
  profile=PF&source=credit_cards&period=2026-07&type=despesa
```

Filtros principais:

Campo	Valores
profile	PF, PJ ou all. Obrigatório por padrão.
source	accounts, credit_cards ou all. Obrigatório por padrão.
period	YYYY-MM. Informe o período ou um intervalo de datas.
date_from / date_to	Datas YYYY-MM-DD. Alternativa a period.
type	receita, despesa, transferencia, cartao_credito ou all.
status	paid, pending, overdue, due_today, future ou all.
spending_type	variavel, fixo, parcelado, normal ou all.
account_id / account_ids	Uma conta ou lista separada por vírgulas.
credit_card_id / credit_card_ids	Um cartão ou lista separada por vírgulas.
category	Nome exato da categoria, sem diferenciar maiúsculas ou acentos.
tag	Nome exato da tag, sem diferenciar maiúsculas ou acentos.

Campo	Valores
description OU q	Trecho da descrição.
paid	true OU false.
include_transfers	true OU false.
page / limit	Paginação; limit de 1..100.
sort	date_desc, date_asc, amount_desc, amount_asc, created_desc OU created_asc.

Também são aceitos os aliases em português perfil, fonte, periodo, data_inicio, data_fim, tipo, situacao, tipo_gasto, categoria, descricao, busca, pago, pagina, limite e ordenacao.

Regras de segurança:

- com `strict_filters=true` — padrão — `profile`, `source` e um período ou intervalo são obrigatórios;
- para pedir tudo, a Nimble precisa enviar literalmente `profile=all` e `source=all`; omitir não significa “tudo”;
- `source=credit_cards&type=despesa` retorna compras no cartão;
- `source=accounts&type=despesa` retorna somente despesas comuns;
- filtro de conta não pode ser combinado com `source=credit_cards`;
- filtro de cartão não pode ser combinado com `source=accounts`;
- nunca misturar `account_id(s)` e `credit_card_id(s)` na mesma chamada.

Resposta resumida:

```
{
  "ok": true,
  "action": "list_transactions",
  "scope": {
    "strict_filters": true,
    "profile": "PF",
    "source": "credit_cards",
    "type": "despesa",
    "period": "2026-07",
    "is_global": false
  },
  "pagination": {
    "page": 1,
    "limit": 50,
    "total_items": 1
  },
  "totals": {
    "income": 0,
    "expenses": 59.9,
    "net": -59.9
  },
  "transactions": [
    {
      "transaction_id": "<TRANSACTION_ID>",
      "source": "credit_cards",
      "type": "cartao_credito",

```

```

        "absolute_amount": 59.9,
        "description": "CapCut",
        "credit_card_id": "<CREDIT_CARD_ID>",
        "profile": "PF",
        "invoice_month": "2026-07"
    }
  ]
}

```

A Nimble deve conferir `scope.profile`, `scope.source` e `scope.period` antes de apresentar os dados. Se o escopo retornado não for o solicitado, não deve responder como se a consulta estivesse correta.

19. POST `settle_transaction`

Marca a pendência como paga/recebida.

POST `/api/v1/whatsapp?action=settle_transaction`

Body:

```

{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "transaction_id": "<TRANSACTION_ID>",
  "confirmed": true,
  "settlement_date": "2026-07-24",
  "notes": "Baixa confirmada pelo usuário"
}

```

Regras:

- `transaction_id` é obrigatório;
- deve vir de `pending_transactions`;
- aceita receitas e despesas comuns;
- ao receber uma perna PF/PJ, baixa as pernas do mesmo `linkedMovementId`;
- ao receber uma perna de transferência interna, baixa o grupo do mesmo `transferId`;
- `settlement_date` é opcional; se omitida, usa a data atual;
- não altera valor, vencimento, conta, categoria ou descrição;
- não aceita `amount`, `account_id`, `category`, `tag`, `date`, `paid`, `new_value`, `new_date`, `undo`, `delete` ou `cancel`;
- não desfaz baixa;
- compras de cartão e pagamentos de fatura não são baixados por esta `action`.

20. GET `payable_invoices`

Consulta faturas e informa se podem ser pagas.

GET `/api/v1/whatsapp?action=payable_invoices&whatsapp_phone=<WHATSAPP_PHONE>`

Campos essenciais:

```
{
  "invoice_ref": "<INVOICE_REF>",
  "ciclo_key": "<CICLO_KEY>",
  "credit_card_id": "<CREDIT_CARD_ID>",
  "credit_card_name": "Nubank",
  "due_date": "2026-07-10",
  "amount": 116.98,
  "remaining_amount": 116.98,
  "status": "FECHADA",
  "can_pay_via_api": true,
  "api_payment_type": "full_only",
  "payment_account_required": true
}
```

Regras:

- somente fatura FECHADA ou ATRASADA é pagável;
- pagamento via API é sempre integral;
- a conta de saída precisa ser escolhida pelo usuário;
- guardar `credit_card_id` e `ciclo_key` para o POST seguinte.

21. POST `pay_credit_card_invoice`

Paga integralmente uma fatura elegível.

POST `/api/v1/whatsapp?action=pay_credit_card_invoice`

Body:

```
{
  "whatsapp_phone": "<WHATSAPP_PHONE>",
  "provider_message_id": "<PROVIDER_MESSAGE_ID>",
  "confirmed": true,
  "credit_card_id": "<CREDIT_CARD_ID>",
  "ciclo_key": "<CICLO_KEY>",
  "account_id": "<ACCOUNT_ID>",
  "payment_date": "2026-07-24",
  "notes": ""
}
```

Regras:

- `credit_card_id`, `ciclo_key` e `account_id` são obrigatórios;
- `payment_date` é opcional;
- se `amount` for enviado, precisa ser exatamente o saldo total;
- pagamento parcial retorna `FULL_PAYMENT_ONLY`;
- fatura aberta/futura retorna `INVOICE_NOT_PAYABLE_VIA_API`;
- a operação cria a despesa bancária e registra o pagamento da fatura;
- em falha intermediária, a API tenta remover registros parciais.

22. GET financial_summary

GET /api/v1/whatsapp?action=financial_summary&whatsapp_phone=<WHATSAPP_PHONE>&period=2026-07&profile=PF

Parâmetros:

- period=YYYY-MM
- profile=PF|PJ
- omitir profile para escopo global
- account_id=<ACCOUNT_ID>
- account_ids=<ACCOUNT_ID>,<ACCOUNT_ID>

Campos principais:

- balances.total_cash_balance: saldo bancário estimado das contas;
- dashboard_summary.current_balance: resultado líquido mensal do período;
- monthly_totals;
- pending_summary;
- overdue_summary;
- due_today;
- upcoming;
- credit_card_summary;
- suggested_messages_for_nimble.

A Nimble não deve chamar total_cash_balance apenas de “saldo atual” sem explicar que é saldo estimado das contas consultadas.

23. GET financial_projection

GET /api/v1/whatsapp?action=financial_projection&whatsapp_phone=<WHATSAPP_PHONE>&months=3&start_period=2026-07&profile=PF

Parâmetros:

- months=1..24 — obrigatório por padrão
- start_period=YYYY-MM
- mode=acumulado|mensal
- profile=PF|PJ|all — obrigatório por padrão
- account_id OU account_ids
- credit_card_id OU credit_card_ids
- include_credit_cards=true|false
- include_transfers=true|false
- strict_filters=true|false — padrão true

Não enviar simultaneamente o singular e o plural do mesmo filtro.

Com filtros estritos, a API retorna FILTER_REQUIRED se a Nimble omitir profile ou months. Para uma projeção global, envie explicitamente profile=all. A resposta repete o escopo em scope.profile, scope.months e scope.period_start; a Nimble deve validar esses campos antes de responder.

Aliases aceitos: `meses`, `periodo_inicial`, `modo`, `perfil`, `incluir_cartoes`, `incluir_transferencias` e `filtros_estritos`.

24. GET financial_analytics

GET `/api/v1/whatsapp?action=financial_analytics&whatsapp_phone=<WHATSAPP_PHONE>&period=2026-07&source=credit_cards&profile=PF&limit=10`

Parâmetros:

- `period=YYYY-MM`
- `profile=PF|PJ|all` — obrigatório por padrão
- `source=general|credit_cards|all` — obrigatório por padrão
- `limit=1..20`
- `account_id` OU `account_ids`
- `credit_card_id` OU `credit_card_ids`
- `strict_filters=true|false` — padrão `true`

Interpretação:

- `source=general`: lançamentos de contas;
- `source=credit_cards`: compras nos cartões;
- `source=all`: retorna as fontes separadas;
- `combined_expenses_total` combina as duas fontes e deve ser explicado dessa forma.

Com filtros estritos, omitir `profile` ou `source` retorna `FILTER_REQUIRED`. Para pedir todas as fontes ou perfis, envie o valor literal `all`. Os aliases `fonte=contas`, `fonte=cartoes/cartões`, `perfil`, `periodo`, `limite` e `filtros_estritos` também são aceitos.

A Nimble deve conferir `scope.profile`, `scope.source` e `scope.period`. Um pedido de “despesas de cartão” precisa usar `source=credit_cards`; um pedido de “despesas comuns” precisa usar `source=general`.

25. Roteamento de intenção para action

Pedido do usuário	Action correta
“Quais são minhas contas/cartões/categorias?”	<code>context</code>
“Crie a categoria Pets na minha PF”	<code>create_category</code>
“Crie a tag Streaming”	<code>create_credit_card_tag</code>
“Lança R\$ 90 de internet”	<code>create_transaction</code>
“Registra uma receita de R\$ 1.200 de serviços”	<code>create_transaction</code> com <code>type:"receita"</code>
“Lança um notebook em 12 vezes na conta”	<code>create_installments</code>
“Registra uma receita de R\$ 3.000 em 3 parcelas”	<code>create_installments</code> com <code>type:"receita"</code>

Pedido do usuário	Action correta
“Lança internet todo mês”	<code>create_fixed</code>
“Registra R\$ 1.200 de receita todo mês”	<code>create_fixed</code> com <code>type:"receita"</code>
“Transfere R\$ 500 do Nubank para o Itaú”	<code>create_transfer</code>
“Lança R\$ 59,90 de CapCut no cartão”	<code>create_credit_card_purchase</code>
“Lança um celular em 12 vezes no cartão”	<code>create_credit_card_installments</code>
“Mostre minhas despesas comuns PF de julho”	<code>list_transactions</code> com <code>profile=PF</code> , <code>source=accounts</code> , <code>period=YYYY-MM</code> , <code>type=despesa</code>
“Mostre minhas despesas de cartão PF de julho”	<code>list_transactions</code> com <code>profile=PF</code> , <code>source=credit_cards</code> , <code>period=YYYY-MM</code> , <code>type=despesa</code>
“Marca a internet como paga”	<code>resolve_transaction</code> → confirmação → <code>settle_transaction</code>
“Pague minha fatura”	<code>payable_invoices</code> → context → <code>pay_credit_card_invoice</code>
“Como está meu financeiro?”	<code>financial_summary</code>
“Qual a projeção PF dos próximos 3 meses?”	<code>financial_projection</code> com <code>profile=PF</code> e <code>months=3</code>
“Onde gastei mais no cartão PF?”	<code>financial_analytics</code> com <code>profile=PF</code> e <code>source=credit_cards</code>

26. Bloco de regras para o agente da Nimble

O texto abaixo pode ser usado como base das instruções operacionais do agente:

Você é o assistente financeiro do FluxMoney no WhatsApp.

A API não é somente de consulta. Você pode criar receitas comuns, parceladas e fixas/recorrentes; despesas comuns, parceladas e fixas/recorrentes; transferências; compras comuns e parceladas no cartão; categorias de receita/despesa; e tags usando as actions oficiais.

Nunca envie `user_id`. Identifique o usuário somente por `whatsapp_phone`.

Antes de criar qualquer lançamento financeiro:

1. consulte `context`;
2. resolva a conta ou o cartão pelo ID retornado;
3. verifique categoria e tag;
4. se a categoria/tag pedida não existir, crie-a pela action apropriada;
5. apresente um resumo completo;
6. aguarde confirmação explícita;
7. execute o POST com `confirmed:true`.

Para pedidos de pagar, receber, baixar ou marcar como pago:

1. consulte `pending_transactions`;
2. selecione o candidato correto;
3. preserve `transactions[].transaction_id`;
4. confirme com o usuário;
5. envie exatamente esse `transaction_id` para `settle_transaction`.

Nunca chame `settle_transaction` sem `transaction_id`.

Nunca invente `account_id`, `credit_card_id`, `transaction_id` ou `ciclo_key`.

Em toda consulta filtrada, traduza a intenção do usuário para parâmetros explícitos. Nunca dependa de valores padrão:

- PF ou pessoal => `profile=PF`;
- PJ ou empresa => `profile=PJ`;
- tudo => `profile=all`;
- despesas comuns/contas => `source=accounts` em `list_transactions` e `source=general` em `financial_analytics`;
- despesas de cartão/fatura => `source=credit_cards`;
- próximos N meses => `months=N` em `financial_projection`;
- mês citado => `period=YYYY-MM`.

Para listar lançamentos, use `list_transactions` e envie sempre `profile`, `source` e `period`, ou um intervalo `date_from/date_to`. Use `type=despesa` para despesas, `type=receita` para receitas e os demais filtros quando mencionados.

Depois de qualquer GET filtrado, confira os campos `scope` da resposta. Só apresente o resultado se `profile`, `source`, `period/months` e IDs coincidirem com o pedido. `FILTER_REQUIRED` significa que falta traduzir um filtro; pergunte ao usuário apenas quando a intenção realmente estiver ambígua.

Use `create_transaction` para lançamento comum.

Use `create_installments` para parcelamento em conta.

Use `create_fixed` para lançamento mensal/recorrente em conta.

Use `create_credit_card_purchase` para compra única no cartão.

Use `create_credit_card_installments` para compra parcelada no cartão.

Use `create_transfer` para transferências e movimentos PF/PJ.

Se uma categoria ou tag não existir, não envie o nome diretamente para a action financeira: crie primeiro com `create_category` ou `create_credit_card_tag`.

Se a API retornar `PENDING_TRANSFER_MATCH_FOUND`, não crie duplicado.

Pergunte se o usuário quer baixar a pendência encontrada ou criar outra.

Edição, exclusão, desfazer baixa, pagamento parcial de fatura e recorrência fixa de cartão não estão disponíveis por esta API.

27. Formato de erro

```
{
  "ok": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "Mensagem",
    "details": {}
  }
}
```


Erros que a Nimble deve tratar:

Código	Tratamento esperado
INVALID_TOKEN	Verificar token na configuração segura.
WHATSAPP_PHONE_REQUIRED	Enviar telefone em toda chamada.
WHATSAPP_NOT_LINKED	Orientar vínculo do número no FluxMoney.
USER_ID_NOT_ACCEPTED	Remover <code>user_id</code> da chamada.
IDEMPOTENCY_KEY_REQUIRED	Enviar header de idempotência.
PROVIDER_MESSAGE_ID_REQUIRED	Enviar ID da mensagem no body.
IDEMPOTENCY_PAYLOAD_MISMATCH	Não reutilizar chave com payload diferente.
CONFIRMATION_REQUIRED	Confirmar com o usuário antes do POST financeiro.
ACCOUNT_ID_REQUIRED	Resolver conta por <code>context</code> .
ACCOUNT_NOT_FOUND	Atualizar contexto e pedir nova escolha.
CREDIT_CARD_ID_REQUIRED	Resolver cartão por <code>context</code> .
CREDIT_CARD_NOT_FOUND	Atualizar contexto e pedir nova escolha.
CREDIT_CARD_INACTIVE	Informar que o cartão está inativo.
CATEGORY_NOT_FOUND	Criar categoria ou pedir outra.
TAG_NOT_FOUND	Criar tag ou pedir outra.
INVALID_AMOUNT	Enviar valor positivo maior que zero.
INVALID_DATE	Enviar data válida em YYYY-MM-DD.
INVALID_BOOLEAN	Enviar booleano real em <code>paid</code> .
INVALID_INSTALLMENTS	Enviar inteiro entre 2 e 120.
INVALID_DEADLINE_MODE	Corrigir modo de recorrência.
FILTER_REQUIRED	Enviar explicitamente perfil, fonte e período/quantidade exigidos pela action.
FILTER_CONFLICT	Remover filtros incompatíveis de conta, cartão, fonte ou tipo.
DATE_RANGE_INVALID	Corrigir o intervalo; a data inicial não pode ser posterior à final.
PENDING_TRANSFER_MATCH_FOUND	Oferecer baixa ou criação explícita de novo movimento.
TRANSACTION_ID_REQUIRED	Reaproveitar ID de <code>pending_transactions</code> .
TRANSACTION_NOT_FOUND	Refazer consulta de pendências.
TRANSACTION_ALREADY_SETTLED	Informar que já está pago/recebido.
UNSUPPORTED_SETTLEMENT_FIELD	Não tentar editar durante a baixa.
FULL_PAYMENT_ONLY	Informar que fatura só pode ser paga integralmente.

Código	Tratamento esperado
INVOICE_NOT_PAYABLE_VIA_API	Informar que a fatura ainda não é elegível.
ACTION_DEPRECATED	Trocar <code>mark_paid</code> por <code>settle_transaction</code> .
ACTION_NOT_SUPPORTED	Orientar uso do painel FluxMoney.

28. Checklist de configuração

- Configurar as 17 actions oficiais listadas na seção 2.
- Não limitar a integração às actions GET.
- Incluir as oito criações financeiras/catalogais:
 - `create_category`
 - `create_credit_card_tag`
 - `create_transaction`
 - `create_installments`
 - `create_fixed`
 - `create_transfer`
 - `create_credit_card_purchase`
 - `create_credit_card_installments`
- Incluir `resolve_transaction`, `settle_transaction` e `pay_credit_card_invoice`.
- Incluir `list_transactions` para consultas detalhadas.
- Configurar Bearer token em armazenamento secreto.
- Enviar `whatsapp_phone` em todas as chamadas.
- Nunca enviar `user_id`.
- Enviar `provider_message_id` e `X-Idempotency-Key` em todo POST.
- Exigir confirmação antes de toda criação financeira.
- Preservar IDs retornados pelas consultas.
- Criar categoria/tag antes do lançamento que depende delas.
- Em projeção, enviar sempre `profile` e `months`.
- Em análise, enviar sempre `profile` e `source`.
- Em listagem, enviar sempre `profile`, `source` e `period` ou intervalo.
- Validar o objeto `scope` antes de responder ao usuário.
- Não configurar `mark_paid` nem `mark_unpaid`.
- Mapear mensagens de erro para respostas amigáveis.

29. Testes mínimos de homologação

Criação comum

- Criar despesa comum com categoria existente.
- Criar receita comum com `type:"receita"` e categoria de receita.
- Tentar criar sem `confirmed:true` e validar `CONFIRMATION_REQUIRED`.
- Repetir mesma chave e mesmo body sem duplicar.

Categoria e tag

- Criar categoria PF de despesa.
- Repetir a criação e validar `already_exists`.
- Criar tag.
- Usar a nova categoria/tag em lançamento.

Cartão

- Criar compra comum no cartão.
- Criar compra parcelada.
- Validar cálculo do mês da fatura.
- Tentar enviar `account_id` e validar bloqueio.

Recorrência

- Criar despesa fixa sem prazo.
- Criar receita fixa sem prazo.
- Criar despesa ou receita fixa com prazo.
- Validar quantidade e datas das ocorrências.

Transferência

- Criar PF → PF como transferência interna.
- Criar PJ → PF como despesa + receita vinculadas.
- Validar bloqueio de duplicidade.
- Baixar uma perna e confirmar baixa do grupo.

Baixa

- Consultar pendências.
- Guardar `transaction_id`.
- Confirmar com o usuário.
- Baixar exatamente o item escolhido.
- Tentar baixar sem `transaction_id` e validar `TRANSACTION_ID_REQUIRED`.

Fatura

- Consultar fatura aberta e confirmar bloqueio.
- Consultar fatura fechada.
- Escolher conta de pagamento.
- Pagar integralmente após confirmação.

Filtros e separação de fontes

- Pedir projeção PF de 3 meses e validar `scope.profile=PF`, `scope.months=3` e três itens em `projection`.
- Repetir a projeção para PJ e confirmar que contas/cartões PF não entram.

- Listar `source=credit_cards&type=despesa` e confirmar que nenhuma despesa comum aparece.
- Listar `source=accounts&type=despesa` e confirmar que nenhuma compra de cartão aparece.
- Resolver uma pendência por descrição e confirmar `selected_transaction.transaction_id`.
- Criar dois lançamentos semelhantes e confirmar `multiple_matches`.
- Filtrar por conta, cartão, categoria, tag, descrição, situação e tipo de gasto.
- Omitir `profile`, `source` ou período nas actions estritas e validar `FILTER_REQUIRED`.
- Enviar filtros de conta/cartão incompatíveis e validar `FILTER_CONFLICT`.

30. Inconsistências encontradas na revisão

Críticas — corrigidas nesta documentação

1. A documentação anterior classificava todas as actions de criação como “legadas/fora do contrato”, embora o código atual as roteie e implemente.
2. O adendo de transferências dizia que `create_transfer` era oficial, enquanto o contrato principal dizia o contrário.
3. A documentação antiga dizia que `pending_transactions` não incluía transferências; o código atual inclui `common`, `internal_transfer` e `pf_pj`.
4. A documentação antiga dizia que `settle_transaction` aceitava apenas lançamentos comuns; o código atual também baixa grupos vinculados por `transferId` e `linkedMovementId`.
5. Faltava ensinar o fluxo “criar categoria/tag primeiro e lançar depois”.
6. As consultas deixavam `profile`, `months` e `source` caírem silenciosamente em `all`/valores padrão quando a Nimble não os enviava.
7. Não existia action detalhada capaz de separar lançamentos comuns de compras no cartão com os demais filtros do sistema.
8. O perfil PF/PJ do cartão era inferido por `brand`, embora o contexto exponha esse perfil em `categoria`; isso classificava cartões PJ incorretamente.

Melhorias recomendadas no backend

1. **Remover `user_id` da resposta de `context`.** O fornecedor não pode enviar nem precisa receber esse identificador.
2. **Criar action para despesa fixa recorrente no cartão.** Hoje `spending_type:"fixo"` em `create_credit_card_purchase` cria apenas uma ocorrência.
3. **Padronizar `paid`.** Em criações de conta ele é obrigatório; em cartão é opcional. Um padrão único reduziria erros de configuração.
4. **Padronizar confirmação de categoria/tag.** O backend não exige `confirmed:true` nessas duas mutações, embora todas as demais criações financeiras exijam.

5. **Usar fuso de São Paulo nas regras de “hoje”.** A função principal de data atual usa UTC e pode divergir do Brasil próximo da meia-noite.
6. **Homologar datas no dia 29, 30 e 31.** Parcelamentos de cartão usam ajuste seguro de fim do mês; recorrências e parcelamentos comuns usam outra rotina e merecem teste específico.

31. Critério de aceite da integração

A configuração só deve ser considerada completa quando o usuário conseguir, pelo WhatsApp:

1. criar uma categoria nova;
2. criar uma categoria de receita e lançar uma receita comum;
3. lançar uma despesa comum em uma categoria de despesa;
4. lançar receita e despesa parceladas em conta;
5. criar receitas e despesas fixas/recorrentes;
6. criar uma tag;
7. lançar uma compra no cartão com categoria e tag;
8. lançar uma compra parcelada no cartão;
9. consultar uma pendência e baixá-la usando o `transaction_id`;
10. repetir uma mesma mensagem sem duplicar o lançamento;
11. obter exatamente três meses ao pedir projeção PF dos próximos três meses;
12. listar despesas de cartão sem misturar despesas comuns;
13. listar despesas comuns sem misturar compras de cartão;
14. filtrar lançamentos por conta/cartão, período, categoria, tag, descrição, situação e tipo de gasto.

Se apenas consultas funcionarem, a integração está incompleta.